

CSE 6220 High Performance Computing
Programming Assignment-3 Report
Submitted April 12, 2025

Name: Honglin Liu
Name: Xuzheng Tian

gID: 903651409
gID: 903618682

1 Implementation of the 2D SUMMA SpGeMM Algorithm

Our implementation of the Sparse General Matrix-Matrix Multiplication (SpGeMM) algorithm computes $C = A \times B$ for sparse matrices A ($m \times p$) and B ($p \times n$) in COO format, using a $2D \sqrt{p} \times \sqrt{p}$ processor grid.

We adopt the Sparse [SUMMA](#) (Scalable Universal Matrix Multiplication) algorithm, optimized for non-zero elements, with key functions implemented in `spgemm.cpp`.

1.1 Algorithm implementation

The algorithm proceeds as follows:

1. Initialization:

The `spgemm_2d` function determines the local block dimensions for each processor.

For a processor at grid position `(row_rank, col_rank)`, we compute block sizes as `block_size_m = (m+size-1)/size`, `block_size_n`, and `block_size_p`, where `size` is the grid dimension ($\sqrt{p}$).

Actual block sizes `(current_size_m, current_size_n, current_size_p)` adjust for boundary conditions.

2. COO to CSR Conversion:

To optimize non-zero element access, `COO2CSR` converts input matrices A and B from COO to CSR format.

For a matrix with `length` non-zeros, `COO2CSR` allocates `row_ptr` (`size m+1`), `idx`, and `values` (`size length`).

It counts non-zeros per row and computes a prefix sum for `row_ptr`, enabling efficient row-wise traversal.

3. SUMMA Computation:

The `SUMMA` function executes the core multiplication.

For each iteration k (0 to `col_size`):

- **Broadcast A :**

The k -th column block of A (CSR format) is broadcast within the row communicator (`row_comm`) using `MPI_Bcast`.

The block includes `row_ptr`, `idx`, and `values`, with size determined by non-zero count (`nnz_A`).

- **Broadcast B :**

The k -th row block of B is broadcast within the column communicator (`col_comm`).

- **Local Computation:**

Each processor computes a contribution to C 's local block (`C_block`, size `block_size_m` $\times$ `block_size_n`).

For each non-zero $A[i, A_col]$ and $B[A_col, B_col]$, we update `C_block[i, B_col] =`

`plus(C_block[i, B_col], times(A[i, A_col], B[A_col, B_col]))`, where `plus` and `times` define the semiring.

`C_block` is initialized to zero in `spgemm_2d` for standard operations.

4. Output Conversion:

The `MTX2COO` function converts `C_block` to COO format. Non-zero elements are extracted, and global coordinates are restored by adding offsets (`row_rank * block_size_m, col_rank * block_size_n`) to the local coordinates. The result is stored in `C`.

1.2 Variations for APSP

For APSP, a variant of `spgemm_2d` described above, *i.e.*, `spgemm_2d_apsp`.

Different from its original version, `spgemm_2d_apsp` initializes `C_block` to 1000000 (infinity, instead of 0) and resets unchanged entries to 0 before conversion, supporting the min-plus semiring employed in APSP.

This variation helps override the bugs in Min-plus *semiring* when using the original `spgemm_2d`, in which case the initialized value, 0, is always being selected as the *minimum value*.

The implementation of both §1.1 & §1.2 ensures correctness for arbitrary $m \times p$ and $p \times n$ matrices (in SpGeMMM) while supporting flexible semiring operations (in ASPs), as tested with $A \times A^T$.

2 Implementation of the APSP Algorithm

The APSP (All-Pairs Shortest Path) algorithm computes shortest path distances between all vertex pairs in a weighted graph with n nodes, represented in COO format.

Inspired by the [Floyd-Warshall](#) algorithm, our implementation in `apsp.cpp` uses matrix multiplication with a *min-plus* semiring operator, executed via `spgemm_2d`.

The graph described in the problem is modeled as an $n \times n$ distance matrix L , where $L[i, j]$ is the edge weight from node i to j (0 denotes infinity).

2.1 Implementation of min-plus semiring

The *min-plus* semiring redefines operations:

- `plus`:
 $\min(a, b)$, selecting the shorter path.
If $a = 0$ and $b = 0$, returns 0; if $a = 0$, returns b ; if $b = 0$, returns a .
- `times`:
 $a + b$, summing edge weights.
If $a = 0$ or $b = 0$, returns 0.

This allows $L^k[i, j]$ to represent the shortest path from i to j using at most k intermediate nodes, akin to Floyd-Warshall's iterative updates.

2.2 Algorithm implementation

The algorithm proceeds as follows:

1. Initialization:

In `apsp`, we set L to the input graph and compute local block ranges for each processor (`row_rank`, `col_rank`) in the $\sqrt{p} \times \sqrt{p}$ grid.

The block size is `block_size_n = (n+row_size-1)/row_size`, with row range `[row_start, row_end)` and column range `[col_start, col_end)`.

2. Iterative Matrix Multiplication:

For iterations where `max_iter` doubles from 1 to n :

- Copy L to L_{tmp} .
- Call `spgemm_2d` to compute $L = L_{\text{tmp}} \times L_{\text{tmp}}$ under the min-plus semiring, updating $L[i, j] = \min(L[i, j], \min_k(L[i, k] + L[k, j]))$.

3. Local Processing:

Post-multiplication, each processor filters L to retain entries within its block, mapping global coordinates (i, j) to local $(i - \text{row_start}, j - \text{col_start})$.

A `std::map` merges `L_tmp` and `L`, keeping the minimum value for each coordinate to ensure correctness.

4. Synchronization:

`MPI_Barrier` ensures processors align after each iteration.

5. Output:

Global coordinates are restored by adding `row_start` and `col_start` offsets, yielding the result in COO format.

This Floyd-Warshall-inspired approach leverages `spgemv_2ds` parallel efficiency to compute shortest paths. The min-plus semiring ensures correctness, while local processing reduces communication overhead.

3 Visualization of Algorithms' Performances